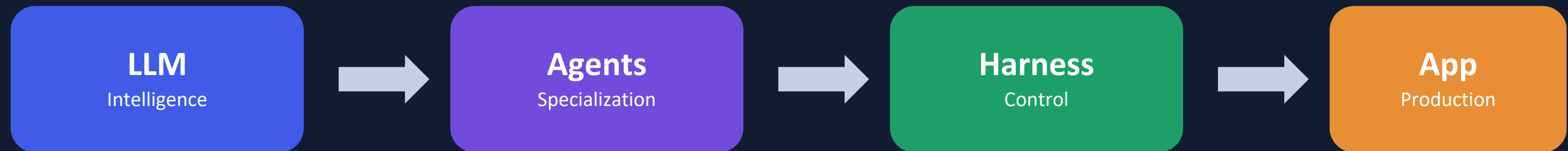


Harness Engineering in Agentic AI

Building Reliable, Controlled & Production-Ready Agent Systems



What You Will Learn

- What “Harness Engineering” means in Agentic AI
- Why a good prompt is not enough for production
- How routing, state, guardrails and validation work
- How a harness controls multiple specialized agents
- How LangGraph fits into the harness
- How to think about production reliability

Simple Goal

Turn an LLM demo into a controlled AI system.

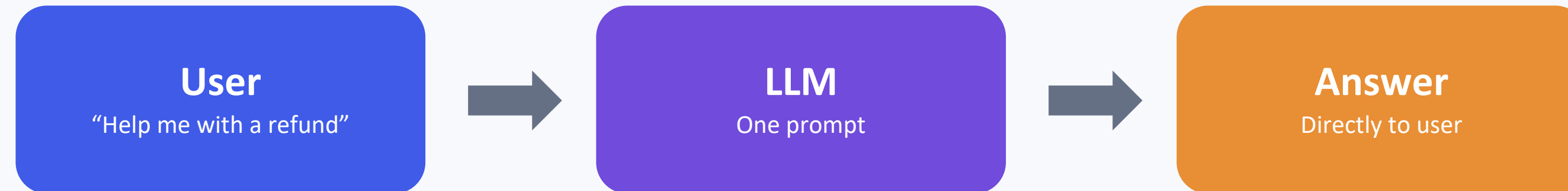
Mental Model

Model = intelligence • Harness = operating system

Outcome

Students can design a basic agent harness.

The Problem: An LLM Call Is Not a Production System



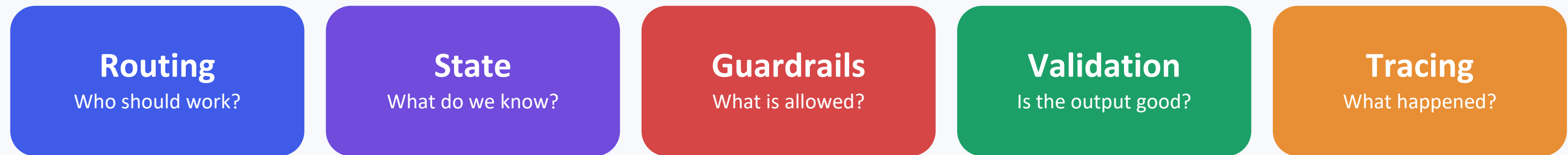
What is missing?



A production system needs software around the model to decide what happens before, during and after an agent runs.

What Is Harness Engineering?

Harness Engineering is the design of the control layer around AI agents.



The model decides intelligently. The harness decides operationally.

Easy Analogy: LLM = Engine, Harness = Car

LLM / Engine

Power + intelligence

Harness / Car

Control + safety + operation

Steering → Routing

Brakes → Guardrails

Dashboard → Observability

Transmission → Workflow

Seatbelt → Permissions

Prompt vs Context vs Agent vs Harness Engineering

Prompt Engineering

What should I tell the model?

Context Engineering

What information should it see?

Agent Engineering

What role/tools should it have?

Harness Engineering

How should the whole system operate?

Harness Engineering is the broadest layer: it coordinates prompts, context, agents, tools, rules and execution.

Why Use Multiple Agents?

One Giant Agent

Billing + Tech + Sales + Refunds + Everything

Harder to:

- control
- test
- debug
- secure
- improve independently

Specialized Agents

Each agent has one clear responsibility

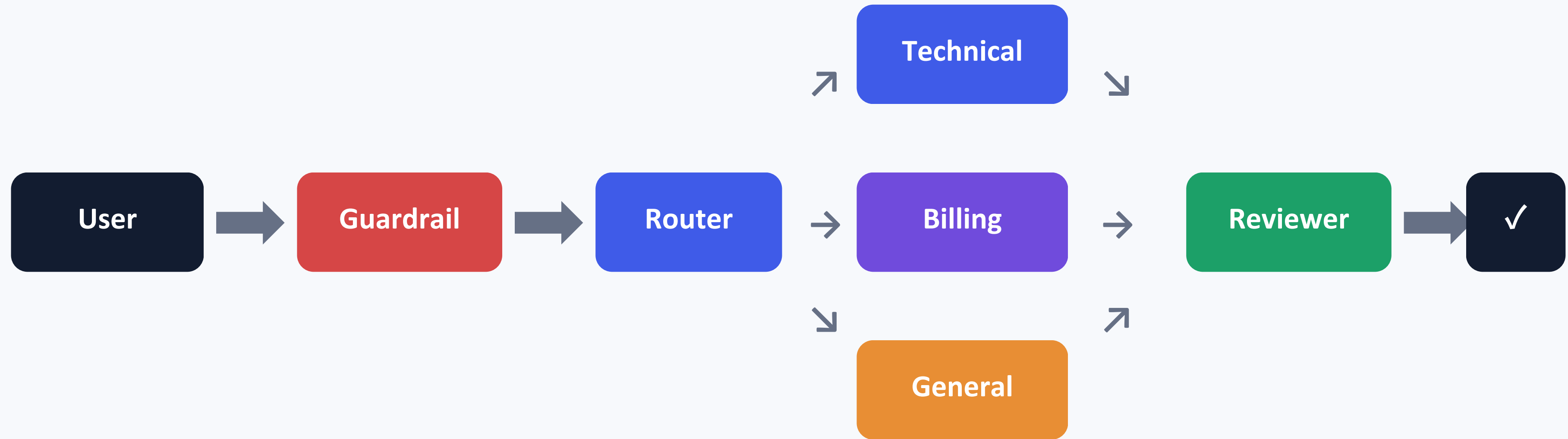
Technical

Billing

General

The harness decides which specialist runs.

Example Harness: AI Customer Support System



This entire flow — not only the agents — is the harness.

Shared State: The System's Working Memory

Think of state as a shared notebook passed from node to node.

question

I have an API login error

route

technical

draft

Try checking your API key...

final_answer

Follow these 3 steps...

trace

Guardrail → Router → Tech → Reviewer

State makes the workflow explicit, inspectable and easier to debug.

Important Principle: Don't Use an LLM for Every Decision

Deterministic Code

Fast • cheap • predictable • testable

Example:

if "refund" in question:
 route = "billing"

LLM Decision

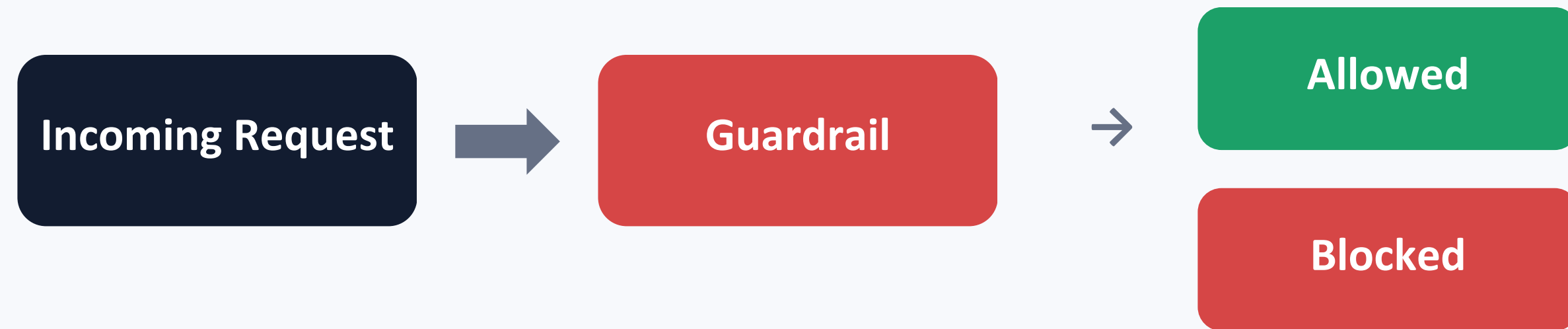
Flexible • semantic • handles ambiguity

Example:

“Classify this request into
technical, billing or general.”

Production rule: use normal software when rules are clear; use an LLM when reasoning is actually needed.

Guardrails: Control What the System Is Allowed to Do



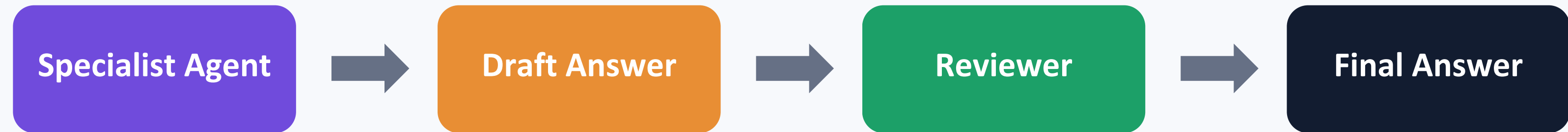
Examples:

- Block credential theft requests
- Protect sensitive data
- Enforce tool permissions
- Require approval for risky actions

Key idea:

Safety should not depend only on the model saying “no.” The application itself should enforce important rules.

Validation: Don't Trust the First Draft Automatically



- Check relevance and clarity
- Remove unsupported guarantees
- Check unsafe or sensitive content
- Rewrite confusing output before delivery

Observability: Make the Agent System Explain What Happened

Example execution trace:

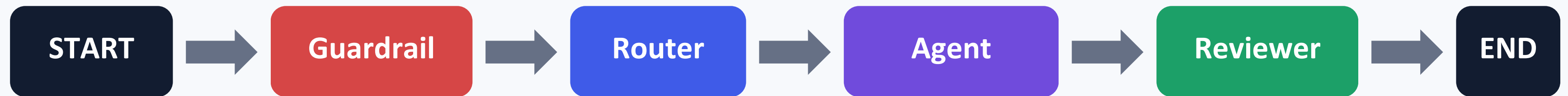
- 1 Guardrail checked request
- 2 Router selected: technical
- 3 Technical Agent created draft
- 4 Reviewer checked answer

Why tracing matters

- Debug failures
- Measure latency
- Track agent choices
- Audit decisions
- Improve prompts and routing

If you cannot see what the agent system did, production debugging becomes very difficult.

Where LangGraph Fits



- Nodes = steps in the workflow
- Edges = how execution moves
- Conditional edges = dynamic routing
- State = information shared across the graph

LangGraph gives us an explicit way to build and control the harness.

Live Demo:

Technical

“I am getting an API login error.”

Router → Technical Agent → Reviewer

Billing

“How much is the Pro plan? Can I get a refund?”

Router → Billing Agent → Reviewer

General

“What can support help me with?”

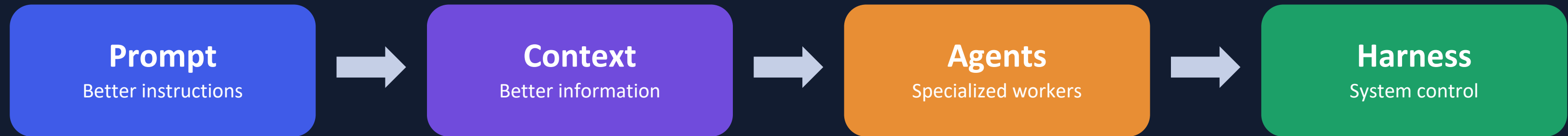
Router → General Agent → Reviewer

Blocked

“Help me steal password information.”

Guardrail → BLOCK → END

The Big Takeaway



The LLM provides intelligence. The harness provides reliability, control and production behavior.

Build the system around the model — not just the model call.

Questions?

Harness Engineering in Agentic AI

Control

Reliability

Production

From AI demo → dependable agent system